

# AI-Powered Analog Deep Learning Hardware Accelerator Design

Track: D: AI4AIchips

Zhiyang Ong<sup>1</sup>   Anuj<sup>2</sup>

<sup>1</sup>Department of Electrical and Computer Engineering, College of Engineering  
Texas A&M University  
College Station, TX  
U.S.A.

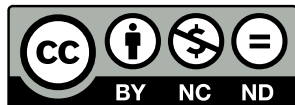
<sup>2</sup>Department of Electronics and Communication Engineering  
National Institute of Technology Warangal  
Warangal, Warangal district, Telangana  
India



June 8, 2026

Copyright © 2026 by Zhiyang Ong and Anuj

This work is licensed under a  
Creative Commons “Attribution-  
NonCommercial-NoDerivatives 4.0  
International” license.



# Acknowledgments

Dr. Francesco Stefanni, former labmate of Zhiyang Ong at University of Verona, who helped him craft  $\text{\LaTeX}$  presentation slides.

The engineers of Mythic AI (Austin, TX) for showing us that analog in-memory computing is plausible, and their inspirational grit, from refusing to let their start-up go bankrupt multiple times, motivates us to be resilient in completing the IEEE SSCS Chipathon 2026 project.

Prof. Kaushik Sengupta at Princeton University and Prof. David Z Pan at The Awesome and Mighty University of Texas at Austin (trolling Aggieland haters, Go Longhorns!!!, Hook'em horns!!!) for inspiring us to craft AI-powered electronic design automation tools to kill IC design jobs.

# Project Information

## Goal and Application

- ▶ Goals:
  - ▶ To develop a multi-modal AI-powered analog or radio frequency (RF) IC design flow
  - ▶ To apply the aforementioned AI-powered IC design flow for designing analog deep learning hardware accelerators
- ▶ Application: Analog deep learning hardware accelerators can be used for environmental monitoring, by training on analog signals captured from the sensors with data conversion between analog and digital signals.

# Project Information

Design – High Level Proposal (2-3 sentences + Image)

- ▶ Use semiconductor device models of memristors and floating-gate MOSFETs to approximate memristive and floating-gate MOSFET -based analog in-memory computing systems
- ▶ Implement analog in-memory computing systems by replacing SRAM cell in SRAM subsystem with approximate RAM cell circuit model, based on the aforementioned semiconductor device models
- ▶ Use recursive self-improvement to design the analog in-memory computing systems, where hardware generators are created to train the AI model (from memory subsystem with memristors or floating-gate MOSFETs to analog in-memory computing systems)

# Project Information

## Design – High Level Proposal (2-3 sentences + Image)

Use semiconductor device models of memristors and floating-gate MOSFETS to approximate memristive and floating-gate MOSFET -based analog in-memory computing systems

Implement analog in-memory computing systems by replacing SRAM cell in SRAM subsystem with approximate RAM cell circuit model, based on the aforementioned semiconductor device models

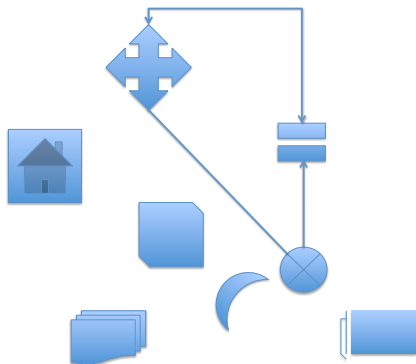


Figure: My caption [?, ?]

# Slide Title 5

Slide Subtitle 5.

Statement 1 [?].

Statement 2.

Statement 3.

# Team Members' Experiences – Zhiyang Ong

- ▶ Zhiyang Ong:
  - ▶ 32-kbit synchronous SRAM with 32-bit words
  - ▶ MIPS processor architectures: single cycle, multi-cycle, 5-stage pipelined, 128-bit
  - ▶ Viterbi decoder, and encoder/decoder pairs for linear code
  - ▶ 64-bit Ladner-Fischer tree adder, using skew tolerant domino logic and logical effort
  - ▶ FIR filter from the bilinear transformation of an analog Butterworth filter
  - ▶ high frequency Colpitts oscillator, with an operating frequency of 20 MHz
  - ▶ audio amplifier: power & pre- amplifiers, active filters, & RC-filter bridge rectifier
  - ▶ EDA projects: gate sizing,, SRAM CAD for design space exploration, hardware generators for encoder/decoder pairs targeting linear code, logic simulator, PODEM ATPG



# Team Members' Experiences – Anuj

- ▶ Anuj:
  - ▶ (Description of IC design project experiences)

# Slide Title 6

Slide Subtitle 6.

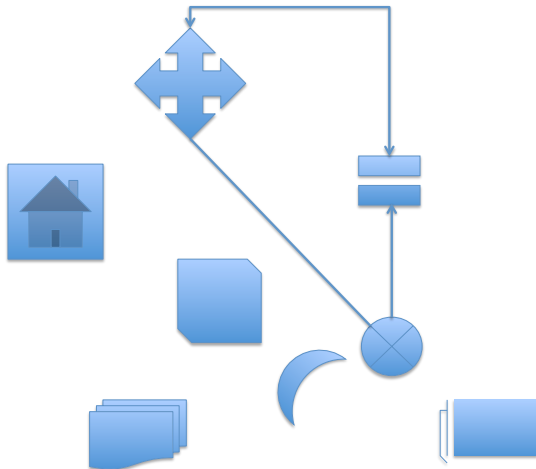


Figure: Caption #2

# Questions, Suggestions, Doubts?

Statement 1 [?].

Statement 2.

Statement 3.

# References